

Leva.Framework.Identity.AspNet

`Leva.Framework.Identity.AspNet` is the ASP.NET Core adapter for `Leva.Framework.Identity`. It connects framework principals and auth sessions to ASP.NET Core authentication, authorization, HTTP cookies, built in endpoints, Google sign-in, and JWT bearer tokens.

ASP.NET Core identity exists so web hosts can use the provider-neutral Identity library through normal ASP.NET Core middleware and endpoint patterns. It does not implement a new identity model, database store, account registration system, profile management, or application-specific authorization rules.

`Leva.Framework.Identity.AspNet` depends on `Leva.Framework.Identity`, `Leva.Framework.Core`, and the ASP.NET Core shared framework. Identity remains provider-neutral and must not depend on ASP.NET Core. Google and JWT support live in this library because, for this framework, they are web-host authentication mechanisms rather than desktop/local identity providers.

```
Leva.Framework.Identity.AspNet
-> Leva.Framework.Core
-> Leva.Framework.Identity
-> Microsoft.AspNetCore
```

Project overview

The ASP.NET Core provider is a host adapter. It reads an `AuthSessionId` from an HTTP cookie or optional request header, loads the framework `AuthSession`, maps the session principal to an ASP.NET Core `ClaimsPrincipal`, and returns a normal ASP.NET Core authentication ticket. That allows applications to use standard ASP.NET Core features such as `UseAuthentication`, `UseAuthorization`, `[Authorize]`, and `RequireAuthorization()` without moving web concepts into the core Identity library.

Login, logout, and principal lookup stay framework-owned. `AspNetIdentityService` calls `AuthenticationService`, writes the created auth session id to the HTTP response, deletes it on logout, and can resolve the current principal/session for custom web endpoints. The built in endpoints are optional convenience endpoints for simple web hosts and samples. Applications can skip them and call `AspNetIdentityService` directly from controllers, minimal APIs, pages, or Blazor endpoints.

Authorization is also delegated back to framework identity. `AspNetAuthorizationRequirement` wraps an `AuthorizationRequirement`, and `AspNetAuthorizationHandler` reads the auth session id from the ASP.NET Core claims principal, loads the framework auth session, and calls `AuthorizationService`. Application-specific ASP.NET Core policies can therefore use the same role, permission, claim, and custom authorization model as non-web hosts.

Google support implements a compact OAuth sign-in flow for ASP.NET Core hosts. It creates a Google authorization redirect, validates the callback state, exchanges the authorization code for an access token, reads Google user information, maps it to a framework principal, and signs the user into a normal framework auth session. Google stays grouped in one feature folder because the defaults, options, user info contract, mapper, policy, sign-in service, and endpoints form one login flow.

JWT support issues and validates compact HMAC-signed bearer tokens for framework auth sessions. It can issue tokens from any configured framework authentication method and can authenticate incoming `Authorization: Bearer` requests through an ASP.NET Core authentication handler. JWT stays grouped in one feature folder because its options, internal mapper, token service, handler, policy, endpoint, and response contract are a cohesive token flow.

Files and classes

Provider composition

`AspNetPrincipalServices` - Registers ASP.NET Core authentication, authorization, auth session adapters, claims mapping, identity services, Google services, JWT services, and HTTP client services for Google token/user-info calls. `AspNetDefaults` - Holds built in ASP.NET Core identity defaults for scheme, cookie, header, and cookie lifetime values. `AspNetIdentityOptions` - Configures authentication scheme name, auth session cookie/header names, optional header support, and cookie behavior.

Authentication implementation

`AspNetAuthenticationOptions` - ASP.NET Core authentication scheme options for framework auth sessions. `AspNetAuthenticationHandler` - ASP.NET Core authentication handler that reads the HTTP auth session id, loads the framework auth session, and creates an authentication ticket. `AspNetClaimsPrincipalMapper` - Maps framework auth sessions and principals to ASP.NET Core claims principals and reads framework auth session/principal ids from claims.

Auth sessions

`AspNetAuthSessionSource` - Resolves the current auth session from the active ASP.NET Core HTTP context for application code and access objects. `AspNetAuthSessionReader` - Reads auth session identifiers from ASP.NET Core request cookies or optional headers. `AspNetAuthSessionWriter` - Writes and deletes auth session identifiers on ASP.NET Core responses. `AspNetIdentityService` - Provides high-level login, logout, principal lookup, and auth session lookup for built in endpoints and custom web UI.

Authorization implementation

`AspNetAuthorizationRequirement` - Wraps one framework `AuthorizationRequirement` for ASP.NET Core policy registration. `AspNetAuthorizationHandler` - Delegates ASP.NET Core authorization checks to framework `AuthorizationService`.

Built in endpoints

`AspNetIdentityEndpoints` - Maps built in `POST /identity/login`, `POST /identity/logout`, and `GET /identity/principal` endpoints. `AspNetLoginRequest` - Request body used by the built in login

and token endpoints. `AspNetLoginResponse` - Response body returned by the built in login endpoint. `AspNetPrincipalResponse` - Response body returned by the built in principal endpoint.

Google sign-in

`AspNetGoogleDefaults` - Holds the built in Google authentication method and endpoint defaults. `AspNetGoogleOptions` - Configures Google client settings, callback path, scopes, correlation cookies, and return URL behavior. `AspNetGoogleUserInfo` - Represents Google user information used to create framework principals. `AspNetGooglePrincipalMapper` - Maps Google user information to a framework `Principal`. `AspNetGoogleAuthenticationPolicy` - Authenticates Google access tokens and creates framework authentication results. `AspNetGoogleSignInService` - Handles Google authorization redirects, local return URL normalization, callback validation, token exchange, and framework login. `AspNetGoogleEndpoints` - Maps built in `GET /identity/google/login` and `GET /identity/google/callback` endpoints.

JWT bearer tokens

`AspNetJwtDefaults` - Holds the built in JWT authentication method, scheme, token type, bearer prefix, and claim defaults. `AspNetJwtOptions` - Configures issuer, audience, signing key, token lifetime, clock skew, and scheme name. `AspNetJwtClaimsMapper` - Internal mapper between framework auth sessions and JWT payload records. `AspNetJwtPayload` - Internal JWT payload record used by framework auth sessions. `AspNetJwtClaim` - Internal framework principal claim record stored in the JWT payload. `AspNetJwtHeader` - Internal JWT header record. `AspNetJwtTokenService` - Issues and validates built in HMAC-signed JWT access tokens. `AspNetJwtAuthenticationPolicy` - Authenticates JWT tokens through the framework authentication service. `AspNetJwtAuthenticationOptions` - ASP.NET Core authentication scheme options for built in JWT bearer tokens. `AspNetJwtAuthenticationHandler` - ASP.NET Core authentication handler that authenticates `Authorization: Bearer` requests. `AspNetJwtEndpoints` - Maps the built in `POST /identity/jwt/login` endpoint for token issuing. `AspNetJwtTokenResponse` - Response body returned by the built in JWT login endpoint.